

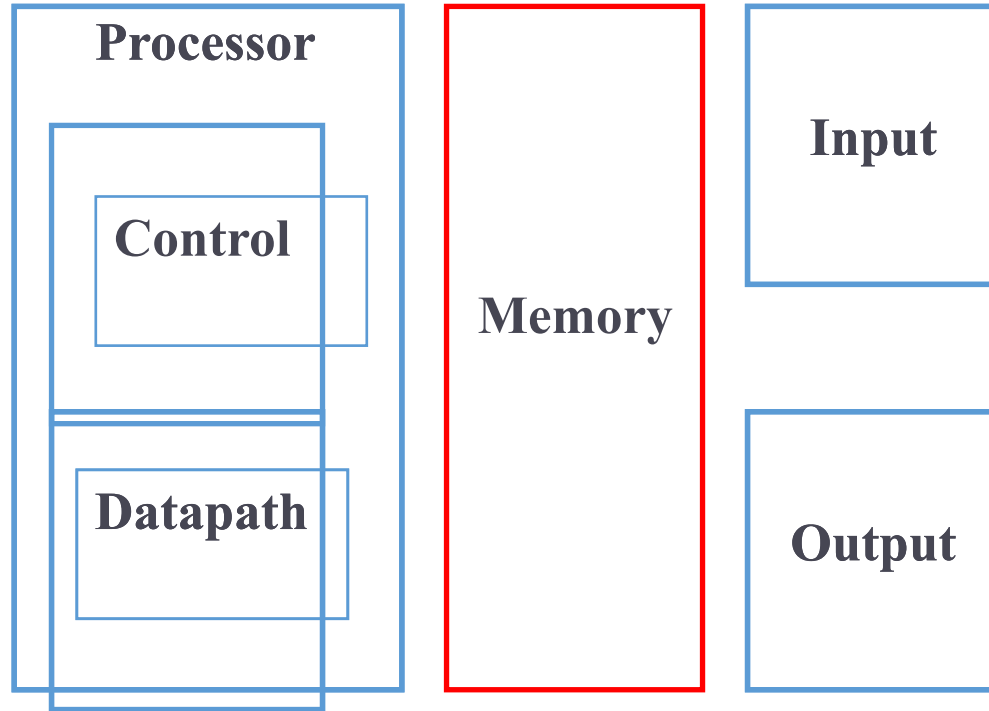


CSE 3313: Computer Architecture

Sidratul Tanzila Tasmi

Lecturer, Department of Computer Science and Engineering
United International University

Components of Computer Memory



Principles of Locality



Programs access a small proportion of their address space at any time

Temporal locality

Items accessed recently are likely to be accessed again soon
e.g., instructions in a loop, induction variables(Fixed inc/dec)

Spatial locality

Items near those accessed recently are likely to be accessed soon
E.g., sequential instruction access, array data

Spatial Locality



Spatial locality refers to the tendency of a program to access data elements that are physically close to each other in memory. When a program accesses a particular memory location, it is likely to access nearby locations soon after.

Arrays: When accessing elements of an array in a loop, spatial locality is exploited as consecutive memory locations are accessed.

```
for (let i = 0; i < array.length; i++) {  
    sum += array[i];  
}
```

Here, `array[i]` accesses consecutive memory locations.

Temporal Locality



Temporal locality, on the other hand, refers to the likelihood of accessing the same memory location multiple times within a short period. This means that if a program accesses a specific piece of data, it is probable that it will access the same data again soon. Caching systems take advantage of this by storing frequently accessed data in faster storage for quick retrieval.

Loops in Code: In a loop, variables are frequently accessed in each iteration, making them good candidates for caching.

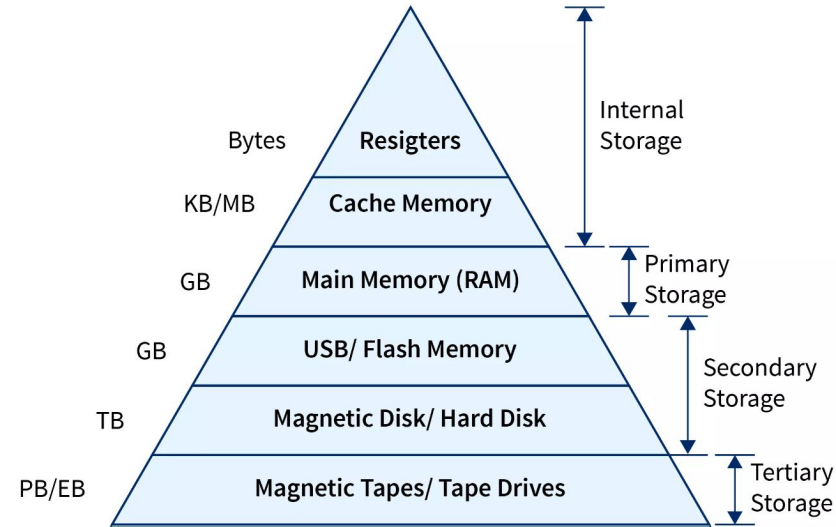
```
for (let i = 0; i < 1000; i++) {  
    sum += array[i];  
}
```

Here, `sum` and `array` will likely be accessed multiple times, so caching them in high-speed memory or registers improves performance.

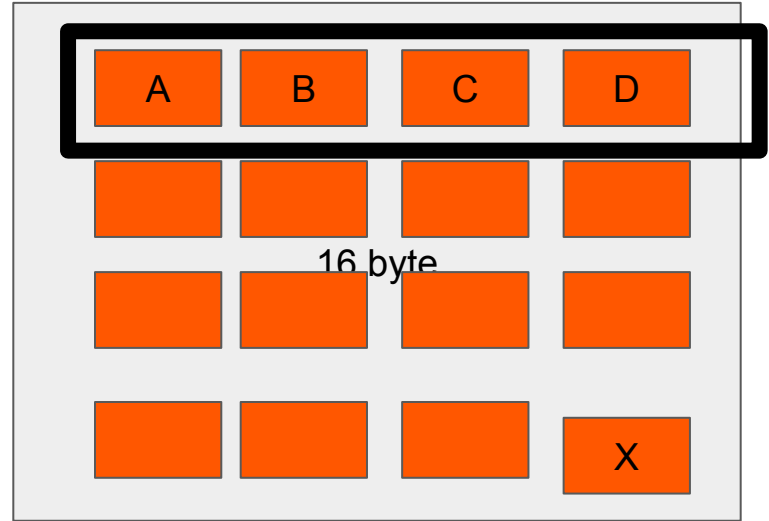
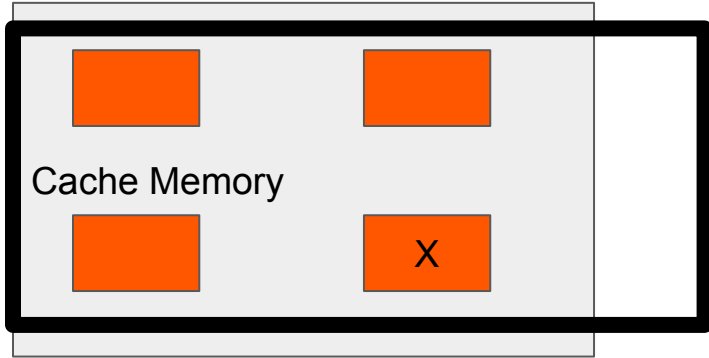
How does hardware exploit principle of locality?



- Organize memory system into a hierarchy
- Multiple levels of memory with different speeds and sizes
- Entire addressable memory space available in largest, slowest memory
- Incrementally smaller and faster memories, each containing a subset of the memory below it, proceed in steps up toward the processor



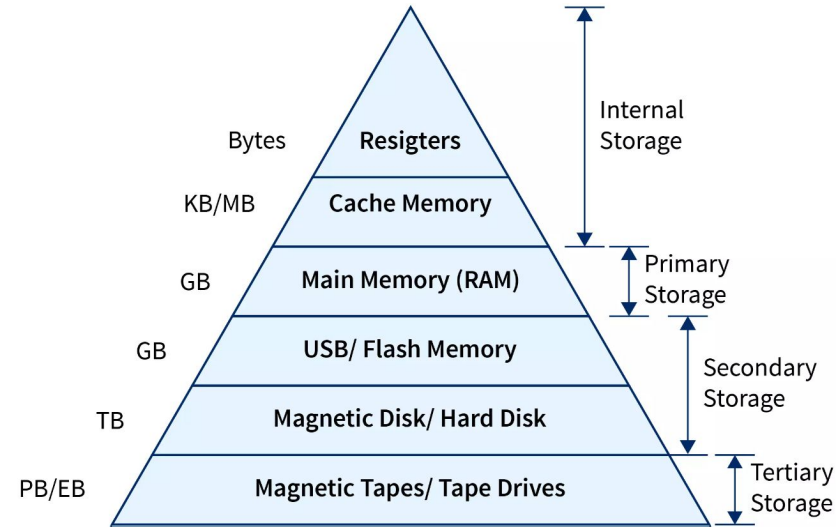
Byte == x? 2s



How does hardware exploit principle of locality?



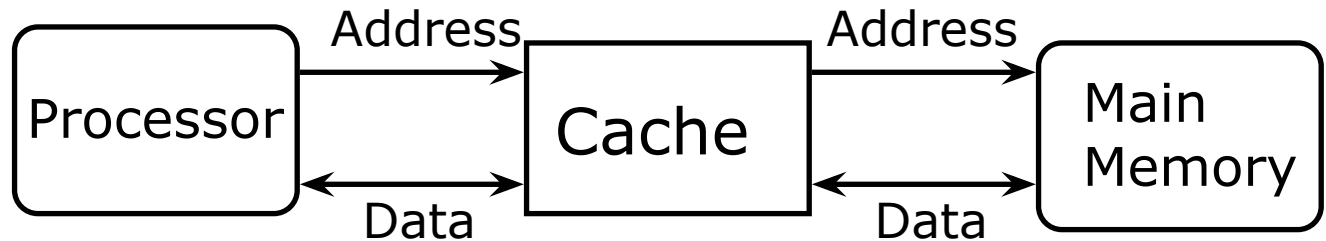
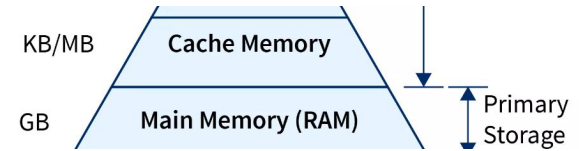
- Store everything on disk
- Copy recently accessed (and nearby) items from disk to smaller **DRAM memory**
 - **Main memory**
- Copy more recently accessed (and nearby) items from DRAM to smaller **SRAM memory**
 - **Cache memory**



Cache Memory



- The level of the memory hierarchy closest to the CPU based on SRAM
- Memorize the most frequently accessed DRAM memory locations to avoid repeatedly paying for the DRAM access latency



Terminologies in Cache Memory

Block / Line



Essentially, a block refers to a group of sequential memory locations treated as a single unit within a cache. This unit is then accessed or transferred as a whole, rather than individual locations being accessed separately.

The minimum unit of information that can be either present or not present in the memory.

The number of memory locations within a block. Larger block sizes can improve performance by bringing in more data at once, but also require more space in the cache.

Cache Hit or Miss



If accessed data is present in upper level

Hit: access satisfied by upper level

If accessed data is absent in upper level

Miss: block copied from lower level



Cache Hit Ratio and Hit Time

Cache hit ratio shows the percentage of requests found in the cache. Calculate the cache hit ratio by dividing the number of cache hits by the combined numbers of hits and misses, then multiplying it by 100.

Cache hit ratio = $\text{Cache hits} / (\text{Cache hits} + \text{cache misses} = \text{Total}) \times 100$

For example, if a website has 107 hits and 16 misses, the site owner will divide 107 by 123, resulting in 0.87. Multiplying the value by 100, the site owner will get an 87% cache hit ratio.

Hit Time: Time to access the upper level which consists of
Access time + Time to determine hit/miss

Miss Ratio and Miss Time



Miss: data needs to retrieve from a block in the lower level

Miss Rate/ Miss Ratio = misses/accesses = 1 - (Hit Rate)

Miss Penalty: Time to replace a block in the upper level + Time to deliver the block to the processor

Direct Mapping



Cache memory

Data is transferred
in block from
main memory to
cache memory



Main Memory

Direct Mapping



Cache memory

Block: A Chunk of Words

Assuming Cache Size = 8 Words

Block Size= 4

Number of Lines in Cache= $8/4=2$



Main Memory size= 32 Words

Block size= 4

Words in each block= $32/4=8$



Direct Mapping

Line 1
Line 2

Cache memory

Block: A Chunk of Words

Assuming Cache Size = 8

Block Size= 4

Number of Lines in Cache= $8/4=2$

Block 1
Block 2
Block 3
...
...
...
Block 7
Block 8

Main Memory size= 32

Block size= 4

Number of blocks = $32/4=8$



Direct Mapping

Line 1
Line 2

Cache memory

Block: A Chunk of Words

Assuming Cache Size = 8

Block Size= 4

Number of Lines in Cache= $8/4=2$

Block 1: w0 w1 w2 w3
Block 2: w4 w5 w6 w7
Block 3: w8 w9 w10 w11
...
...
...
Block 7: w24 w25 w26 w27
Block 8: w28 w29 w30 w31

Main Memory size= 32

Block size= 4

Number of blocks = $32/4=8$



Direct Mapping

Line 1
Line 2

Cache memory

Number of bits required to represent memory:

Total words= 32

To represent 32 bits, bit required is
 $\log_2 32 = 5$

Block 1: w0 w1 w2 w3

Block 2: w4 w5 w6 w7

Block 3: w8 w9 w10 w11

...

...

...

Block 7: w24 w25 w26 w27

Block 8: w28 w29 w30 w31



Direct Mapping

Line 0
Line 1

Cache memory

Memory Block Assignment:

$i = j \text{ modulo } m = j \% m$

where,

i = cache line number

j = main memory block number

m = number of lines in the cache (2)

Block 0: w0 w1 w2 w3

Block 2: w4 w5 w6 w7

Block 3: w8 w9 w10 w11

...

...

...

Block 6: w24 w25 w26 w27

Block 7: w28 w29 w30 w31



Direct Mapping

Line 0: Block 0
Line 1

Cache memory

Memory Block Assignment:

$i = j \text{ modulo } m = j \% m$

where,

i = cache line number

j = main memory block number = 0

m = number of lines in the cache (2)

Block 0: w0 w1 w2 w3

Block 2: w4 w5 w6 w7

Block 3: w8 w9 w10 w11

...

...

...

Block 6: w24 w25 w26 w27

Block 7: w28 w29 w30 w31

$i = 0 \% 2 = 0$



Direct Mapping

Line 0: w0 w1 w2 w3
Line 1

Cache memory

Memory Block Assignment:

$i = j \text{ modulo } m = j \% m$

where,

i = cache line number

j = main memory block number = 0

m = number of lines in the cache (2)

Block 0: w0 w1 w2 w3

Block 2: w4 w5 w6 w7

Block 3: w8 w9 w10 w11

...

...

...

Block 6: w24 w25 w26 w27

Block 7: w28 w29 w30 w31

$i = 0 \% 2 = 0$



Direct Mapping

Line 0: w0 w1 w2 w3
Line 1: Block 3

Cache memory

Memory Block Assignment:

$i = j \text{ modulo } m = j \% m$

where,

i = cache line number

j = main memory block number = 0

m = number of lines in the cache (2)

Block 0: w0 w1 w2 w3

Block 2: w4 w5 w6 w7

Block 3: w8 w9 w10 w11

...

...

...

Block 6: w24 w25 w26 w27

Block 7: w28 w29 w30 w31

$i = 3 \% 2 = 1$



Direct Mapping

Line 0: Block 6
Line 1: Block 3

Cache memory

Memory Block Assignment:

$i = j \text{ modulo } m = j \% m$

where,

i = cache line number

j = main memory block number = 0

m = number of lines in the cache (2)

Block 0: w0 w1 w2 w3

Block 2: w4 w5 w6 w7

Block 3: w8 w9 w10 w11

...

...

...

Block 6: w24 w25 w26 w27

Block 7: w28 w29 w30 w31

$i = 6 \% 2 = 0$



Direct Mapping



Memory Block Assignment Formula:

Line Number = Block Number % Total Number of Lines

Direct Mapping

Line 0: Block 6
Line 1: Block 3

Cache memory

Block 0: w0 w1 w2 w3
Block 2: w4 w5 w6 w7
Block 3: w8 w9 w10 w11
...
...
...
Block 6: w24 w25 w26 w27
Block 7: w28 w29 w30 w31

Main Memory



Physical Address Representation of Main Memory



The Main Memory consists of memory blocks and these blocks are made up of fixed number of words. A typical address in main memory is split into two parts:

Index Field: It represent the block number. Index Field bits tells us the location of block where a word can be.

Block Offset: It represent words in a memory block. These bits determines the location of word in a memory block.

1

Index	Block Offset
	0/1/2/3

Physical Address Representation of Main Memory



Index Field: It represent the block number. Index Field bits tells us the location of block where a word can be.

Block Offset: It represent words in a memory block. These bits determines the location of word in a memory block.

Block 0: w0 w1 w2 w3
Block 1: w4 w5 w6 w7
Block 2: w8 w8 w10 w11
Block 3: w12 w13 w14 w15

Index	Block Offset
	2 (To represent 4 places, 2 bit is required)

Physical Address Representation of Main Memory



Index Field: It represent the block number. Index Field bits tells us the location of block where a word can be.

Block Offset: It represent words in a memory block. These bits determines the location of word in a memory block.

Block 0: w0 w1 w2 w3
Block 1: w4 w5 w6 w7
Block 2: w8 w8 w10 w11
Block 3: w12 w13 w14 w15

Index	Block Offset
Block 0/ Block 1/ Block 2. Block 3	2 (To represent 4 places, 2 bit is required)

Physical Address Representation of Main Memory



Index Field: It represent the block number. Index Field bits tells us the location of block where a word can be.

Block Offset: It represent words in a memory block. These bits determines the location of word in a memory block.

Block 0: w0 w1 w2 w3
Block 1: w4 w5 w6 w7
Block 2: w8 w8 w10 w11
Block 3: w12 w13 w14 w15

Index	Block Offset
2 (To represent 4 blocks)	2 (To represent 4 places, 2 bit is required)

Physical Address Representation of Main Memory



Index Field: It represent the block number. Index Field bits tells us the location of block where a word can be.

Block Offset: It represent words in a memory block. These bits determines the location of word in a memory block.

Block 0: w0 w1 w2 w3
Block 1: w4 w5 w6 w7
Block 2: w8 w8 w10 w11
Block 3: w12 w13 w14 w15

Index	Block Offset
2	2
Total Bit Required= 2+2=4	

Physical Address Representation in Cache Memory



The Cache Memory consists of cache lines. These cache lines has same size as memory blocks. The address in cache memory consists of:

Block Offset: This is the same block offset we use in Main Memory.

Index: It represent cache line number. This part of the memory address determines which cache line (or slot) the data will be placed in.

Tag: The Tag is the remaining part of the address that uniquely identifies which block is currently occupying the cache line.

Important Formulas



Number of Blocks = Physical Memory Size / Block Size
Number of Lines in Cache = Cache Memory Size / Block Size

Important Formulas



Physical Memory Bit Size = $\log_2$ (Physical Memory Size)

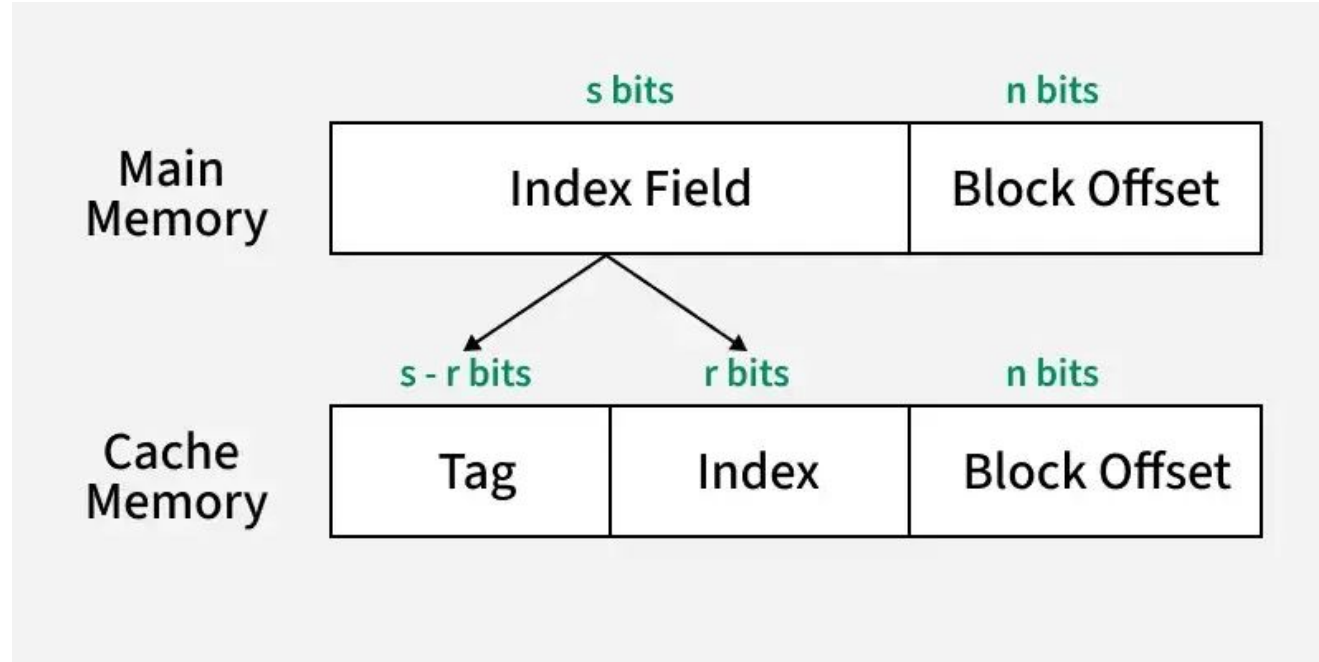
Index Field to determine Block = $\log_2$ (Total No of Blocks)

Index Field to determine Cache Line Number = $\log_2$ (Total No of Lines in Cache)

Block Offset = $\log_2$ (Block Size)

Tag Bit Size = Index Field for block - Index field for cache line

Memory Structure in Direct Mapping



Solved Example



Consider a direct mapped cache of size 16 Bits with block size 4 bits. The size of main memory is 128 bits. Find-

Number of bits in tag

Solved Example



Consider a direct mapped cache of size 16 Bits with block size 4 bits. The size of main memory is 128 bits. Find-

Number of bits in tag

Tag directory size

Size of main memory= $\log_2 X = 128$

$X = 7$

Block offset= $\log_2 Y = 4$, $y = 2$

Index of main memory = $7 - 2 = 5$

Solved Example



Cache Memory

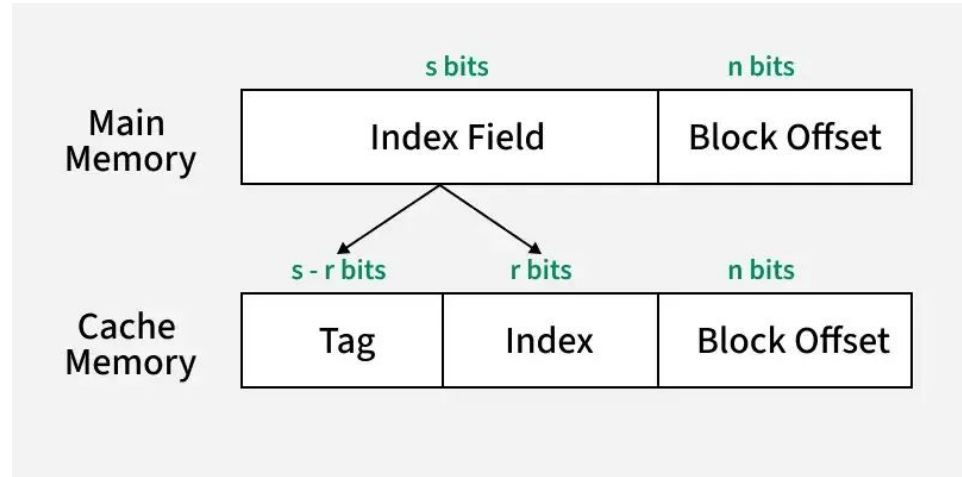
Index= Line Number

Line number of cache memory=

Cache size/ Block size= $16/4=4$

Main Memory Index Field= Index
Field of cache Memory + Tag

Tag= Main memory index field -
field size of cache memory= $5-4=1$





Thank you!